Forge-LLM + tern158

Plan to merge the 1.58-bit ternary project into Forge-LLM and make Forge the all-inclusive training, quantization and fine-tuning system

Document status	DRAFT v1 for owner review. Nothing in either repository has been modified by this plan.
Date	2026-11-01
Source project	tern158 v0.1 (pre-release) at `C:\develelement\1 58b\NEW\CUDA-1.58B`
Target project	Forge-LLM (branch main, HEAD 959940b) at `C:\develelement\FORGE LLM`
Basis	Read-only survey of both code bases (6 parallel readers, about 180 file reads), the two hardware log bundles from 2026-10-27, and Forge's own docs/ADRs.
Next step	You review. Section 14 lists the decisions I need from you. Work starts only after your review.

How to read this. Section 1 is the two-page summary. Sections 4 to 9 are the technical design (structure, kernels, training, quantization, platform, UI). Section 10 is the phased roadmap with exit criteria. Section 14 is the checklist of open decisions. The appendices map every tern158 file to its destination.

Contents

1	Executive summary
2	What each project brings (and what is proven vs unproven)
3	Design principles and key decisions (draft ADR 0007)
4	Target file structure and migration map
5	Kernel plan
6	Training integration (method layer, QAT, sharded trainer)
7	Quantization, conversion, fine-tuning and serving
8	Platform: jobs, API, CLI, planner, eval gates, telemetry
9	UI additions
10	Phased roadmap with exit criteria
11	Testing and validation plan
12	tern158 defects and gaps to fix during the port
13	Risk register
14	Open decisions for your review
A	File-by-file migration map
B	Config key mapping (TOML to pydantic)
C	Measured results carried over from hardware logs
D	Extension-point checklists

1 Executive summary

Goal. Fold tern158, the 1.58-bit {-1, 0, +1} training/conversion/fine-tuning project, into Forge-LLM so that Forge is a single system covering training, quantization, fine-tuning and serving across every precision it supports, ternary included. tern158 as a standalone repo is retired once the merge is verified.

What we are merging. tern158 is about 7,800 lines: 2,570 lines of CUDA (quantisers, 2-bit packing, four int8 ternary GEMM backends with start-up self-validation, fused norm/quantise, loss and optimizer kernels), 4,330 lines of Python (ternary trainer with sharded latent weights and packed-code all-gather, HF conversion with SubLN, warm-up and KL distillation, health monitor, simulator, orchestrator, UI) and 219 lines of CPU tests. On an RTX PRO 6000 Blackwell (sm_120) it passed 23/23 kernel selftests and 65/65 Python selfchecks, reached 346-411 TOPS on the int8 mma path, and trained a 125M-parameter ternary model on 200M tokens in 19.7 minutes (about 170k tokens/s, val perplexity 141 at step 500, zero-weight fraction 40.4%).

Core approach. Add ternary as one more method on Forge's existing extension points rather than transplanting tern158 wholesale. Concretely:

- **Reuse Forge's platform.** Job manager, SSE/SQLite telemetry, schema-driven forms, store, serving engine, planner, gates and claims ledger replace tern158's orchestrator, stdlib UI server, run-directory protocol and GPU table.
- **Make ternary a first-class format.** New linear kinds `t158` (packed, inference) and `fq_t158` (latent-weight QAT), a ternary layout registered in Forge's layout system, an oracle ported from tern158's `ref_ops.py`, and a Forge-native artifact that loads and serves with no engine change.
- **Triton-first kernels, native CUDA kept as an optional tier.** Per ADR 0002 the ternary GEMM and quantiser are ported to Triton and validated against tern158's measured CUDA numbers. The proven hand-written mma.sync kernel moves to `forge/native/ternary` as an optional extension behind a validation probe, so nothing is lost if Triton falls short.
- **Introduce a Method layer.** A small catalog (`forge.methods`) describes every training, fine-tuning and quantization method (bf16 full FT, LoRA/QLoRA/DoRA, FP8, NVFP4, PTQ family, QAT, distillation, ternary) with its capabilities, diagnostics pack and simulator scheme. It powers a Method picker in the UI, the planner and the training simulator, so future methods (and ternary) plug in the same way.
- **Generalise tern158's unique assets.** The training memory/time simulator becomes a Forge planner module for all schemes, the health monitor becomes a per-method diagnostics framework, and the report-derived rules become graded claims and gates (g-t158) labelled TARGET/untested until Forge itself measures them.

Shape of the work. Eight phases (Section 10). P1 to P3 are CPU-testable on this Windows machine and deliver a ternary artifact that trains (small scale), converts, loads, serves and evaluates. P4 (kernels) and P5 (sharded multi-GPU trainer) need a rented GPU and are gated by the validation ladder carried over from tern158 (G0 to G3). The UI work in P6 is mostly additive because the New Job form is already generated from pydantic schemas.

> **Honesty constraint.** Forge's docs state that no Forge GPU path has been validated except on one sm_120 box, and tern158 has validated only single-GPU sm_120. Everything in this plan that touches multi-GPU, sm_80/86/89/90/100/103, or Triton ternary performance is therefore a hypothesis until measured. The plan bakes in explicit gates so no speed or quality claim reaches the docs, UI or claims ledger without a measurement.

Decisions I am asking you to confirm (full list in Section 14)

#	Decision	My recommendation
D1	Triton-only vs keep native CUDA	Triton-first (ADR 0002 stands); keep tern158's CUDA as optional `forge/native/ternary` tier. Needs new ADR 0007.
D2	Dedicated job type vs precision rung inside train/finetune	Both, sharing one core: `ternary.*` job types for the dedicated flows, plus a QAT method selectable from train/finetune.
D3	Keep `.t158.safetensors`	Yes as import/export only; Forge directory artifact is canonical.

#	Decision	My recommendation
D4	Package name for native build	`forge_ternary_C`; env vars map to FORGE_*.
D5	Sequence after the in-flight CUDA-compat work	Commit that work first (14 modified + 7 untracked files), then branch `ternary-merge`.

2 What each project brings

2.1 tern158: accomplishments to preserve

Area	What exists	Evidence level
CUDA kernels	Quantisers (absmean, TWN, Lloyd; per-tensor/row/group), 2-bit pack/unpack, per-token int8 activation quant, fused RMSNorm+quant, Hadamard (16..8192), 4 GEMM backends (dp4a GEMV, wmma, mma.sync m16n8k32 with 3-stage cp.async, bf16+cuBLAS), SwiGLU, RoPE, fused CE and KD-KL, AdamW with stochastic-rounded bf16 state, norm-clip helpers.	Measured on sm_120: 23/23 selftest, 65/65 selfcheck.
Self-validation	`t158_validate_backends`: 8 shape cases vs fp64 host reference, NaN-fill, bitmask result, automatic mma>wmma>dp4a fallback; sm_103 routed to bf16 because int8 pipe is about 1/30 of FP8.	sm_120 only; sm_103 rule is datasheet-derived.
Trainer	Scratch / convert / finetune (full or LoRA). Row-sharded latent weights + Adam state, reduce-scatter of dW, packed 2-bit code all-gather (8x less than bf16), resumable checkpoints across GPU counts, WSD/cosine/two-stage schedules, wd to zero in cooldown, warm-up lambda, KL distillation.	Single-GPU measured; multi-GPU gloo-tested on CPU only.
Conversion	HF Llama/Mistral/Qwen-like import: qkv and gate/up fusion, SubLN (gain 1), Lloyd init, HF-style lambda warm-up, KD from frozen bf16 teacher; per-layer reconstruction SQNR report.	Code-complete; not hardware-validated end to end.
Health	Zero density (healthy 40-52%, warn 60%, stop 80%), flip rate vs predicted equilibrium 0.59*sqrt(2*lr*wd), loss-spike detector, cloze probe, NaN stop.	Observed: zero 40.4%, flip 0.06-0.69%.
Simulator	Memory per GPU, step-time roofline, comm volume, MFU, cost, energy, advice. ~30 GPUs in table.	Uncalibrated first-order model.
Tooling	doctor, build, selftest, nccl_test, vast.ai scripts, log-bundle collector with token redaction, 10 TOML presets.	Used in two real sm_120 runs.
Research rules	docs/REPORT_NOTES.md: never learn the scale, Lloyd step, SubLN, stochastic rounding below lr 3e-4, G=128 for converted models, etc., with evidence grades A-D.	Literature-derived, mostly unreproduced here.

2.2 Forge-LLM: what we plug into

Layer	State today	Relevance to ternary
Kernels	Triton-only (no .cu/.cpp). 5 specs; ops_gemm.py dispatch with oracle fallback; layouts (w4, w8, fp4) with hashed ids and an unused `native` executor hook; autotune cache; W4 M-routing (gemv / dot / dequant+cuBLAS).	int8 tl.dot with int32 accumulate already exists (gemm_int8.py); ternary = 2-bit unpack in-kernel + same dot.
Linears	Registry (`register_linear`), PackedLinear contract (buffers, from_dense, from_shape, dequantize, quant_spec); fake-quant QAT linears (fq_int, fq_nvfp4) with STE.	Direct templates for `t158` and `fq_t158`.
Training	One Trainer (pretrain/finetune/distill), precision ladder R0/R1/R2 by class-swap, DDP/FSDP only, single torch.save checkpoint, AdamW/8-bit/Lion/SGD.	No hooks for custom optimizer step, sharded latents or packed all-gather: needs a plug-in refactor.
Compress	13 recipe stages, PTQ (rtn, gptq, awq, smoothquant, mrgptq), rotation/Hadamard, prune, lowrank, BitDelta, KV quant, lossless.	Add ternarize / ternary_qat / pack branch; reuse Hessian machinery.

Layer	State today	Relevance to ternary
Platform	JobManager + SQLite, SSE metrics, schema-driven forms, store, serve engine, planner (inference-oriented), eval gates, claims ledger, 16 UI routes.	Ternary appears in New Job automatically once a job type is registered.
Hygiene	Honest validation tables (new rows start at 'no'), provenance tags, ADRs, CPU-testable by design. 111 CPU test files + 18 GPU.	We follow the same discipline.

Working tree note. Forge currently has uncommitted CUDA-compat work (`forge/hw/cuda_compat.py`, `forge/jobs/cuda_gate.py`, edits to cli.py, worker.py, three UI files, scripts, docs/KERNELS.md). It does not conflict with kernels or formats, but the merge edits several of the same files. Plan: you commit it (or tell me to), then we branch.

3 Design principles and key decisions

These are the proposed contents of ADR 0007 ("Ternary and method layer"). Each is a deliberate choice you can overturn at review.

#	Principle	Consequence
1	**Additive, not a transplant.** Ternary uses Forge extension points (linear registry, job registry, stage registry, kernel specs, gates, claims).	tern158's monolithic TernaryLM and orchestrator are not copied; their logic is re-homed.
2	**Triton-first, torch oracles, native optional.** Same rule as ADR 0002.	Ternary GEMM ships first as oracle + Triton; the proven CUDA mma kernel is an optional executor selected only if its probe passes and it measurably wins.
3	**One format definition.** Numerics live once in `forge/formats/ternary.py` (quantisers, act quant, STE), used by PTQ, QAT, oracle and tests.	Same principle as ADR 0005 for FP4.
4	**QAT module and packed module are separate classes.** Latent weight is a Parameter; packed weight is buffers.	`fq_t158` for training, `t158` for inference; `from_dense` converts.
5	**Bit-compatible with tern158 storage.** tern158 packs 16 codes per little-endian int32 (code j at bits [2j+1:2j]; 00=0, 01=+1, 11=-1). That is byte-identical to Forge's `pack_bits(codes, 2)` when codes = t & 3 (checked by reading both implementations; golden test still required; pack_bits pads K to a multiple of 4 while tern158 requires K divisible by 16).	No repack needed between Forge portable layout and tern158 kernels; the layout is registered as `ternary2_row` with its own hashed id.
6	**Method layer is metadata + wiring, not a rewrite.** `forge.methods` is a registry of descriptors pointing at existing job types, stages and diagnostics.	Delivers 'all-inclusive' navigation and planning without destabilising the Trainer.
7	**Honesty gates.** Unvalidated GPU paths are behind explicit flags (like `allow_unvalidated_fp4`) and labelled EXPERIMENTAL; numbers from papers are TARGET/untested claims.	Nothing from tern158's report becomes a Forge 'fact' without a Forge measurement.
8	**Safe persistence.** Latent shards go to safetensors, not pickle; loaders use `weights_only` where torch.load remains.	Fixes a tern158 weakness before it enters a platform that accepts uploads.

Target architecture: Forge-LLM after the merge (green = new, blue = existing/extended, amber = optional native)

UI (vanilla JS, no CDN)	Method picker + Ternary Lab	Job detail: health panels	New Job / Fine-tune (schema-driven)	Models / Kernel Lab / Hardware / Planner
API + jobs	job types: ternary.*	planner: train-sim + Scheme t158	jobs / metrics / SSE (unchanged)	cuda_gate (reused)
Methods + recipes	forge.methods catalog registry	stages: ternarize, ternary_qat, pack	PTQ / distill / finetune stages	gates: g-t158 claims, rules
Training + models	forge.ternary QAT, health, convert	zero_ternary strategy (sharded latents)	Trainer / Precision (plug-in refactor)	linear registry: t158, fq_t158
Kernels + formats	formats/ternary layouts/ternary	Triton ternary GEMM + quantiser	ref oracles (ported ref_ops)	forge/native/ternary CUDA ext (optional)
Hardware + store	ChipSpec + int8 flag (one chip table)	cuda_compat (+ native build)	artifact store kind=t158	.t158.safetensors import/export
Retired after migration:	tern158 ui/server.py + index.html	orchestrator.Jobs, pid/control.json	gpus.py table, data.py loader	TOML configs (become YAML)

4 Target file structure and migration map

New code goes where Forge already keeps that kind of thing. Only ternary-specific training logic gets a new package. Items marked (new) do not exist today; (ext) extends an existing file.

```
forge/
  methods/                    (new) catalog: MethodSpec registry + descriptors for ALL methods
    __init__.py  catalog.py  builtin_methods.py  diagnostics_packs.py
  ternary/                    (new) ternary-specific training/conversion code
    config.py                 pydantic TernaryConfig (from tern158 dataclasses, ui() hints)
    qat.py                    TernaryQATLinear (fq_t158), _TernaryFn STE, warm-up fn
    optim.py                  latent-weight AdamW, schedules (wsd/twostage), wd-to-zero, SR state
    zero.py                   zero_ternary strategy: sharded latents, reduce-scatter, packed all-gather
    convert.py                HF -> ternary (qkv/gate-up views, SubLN, Lloyd init, SQNR report)
    export.py                 Forge artifact writer + .t158.safetensors import/export
    health.py                 ternary diagnostics pack (zero density, flip rate, collapse alarms)
    run.py                    job entrypoints: ternary.train / convert / finetune / export / selftest
    stages.py                 recipe stages: ternarize, ternary_qat
    presets.py                size presets (125m..70b) -> pydantic
  formats/ternary.py          (new) absmean/TWN/Lloyd quantisers, per-token int8 act quant, STE
  formats/spec.py             (ext) format_info + fake_quantize branch 'ternary'
  kernels/
    specs/ternary_w158a8_gemm.yaml        (new)
    ref/gemm_ternary.py                   (new) port of ref_ops.py (oracle)
    layouts/ternary.py                    (new) PackedTernary + layout 'ternary2_row'
    triton/gemm_ternary.py                (new) GEMM + GEMV variants
    triton/quant_ternary.py               (new) segment quantiser + packer
    ops_gemm.py                           (ext) ternary_linear, KernelT158Linear
    catalog.py, bench.py, ops.py          (ext) feature gate, bench case, _DOCUMENTED
    layouts/chip.py                       (ext) has_int8, int8_tc_fast
  native/ternary/             (new, OPTIONAL) kernel/*.cu, bindings, CMake/setup, selftest.cu, stub headers
  models/linear.py            (ext) register fq_t158; models/loader.py (ext) kinds + suffix list
  models/config.py            (ext) ternary_subln, per-group linear-kind map
```

```
  compress/ptq.py, stages.py  (ext) format='ternary'; compress/packed.py (ext) T158Linear
  train/precision.py          (ext) plug-in quant-mode interface (R1/R2/ternary as peers)
  train/dist.py, trainer.py   (ext) strategy hook: 'zero_ternary'; checkpoint shard-aware
  planner/trainsim.py         (new) training memory/time/cost simulator, all schemes
  planner/spec.py, rulebase.py, claims_seed.py   (ext) Scheme t158, R-Q7-TERNARY, claims
  hw/chips.py, hw/probe.py    (ext) one chip table (+tdp, pcie, int8 flag); ternary probe check
  eval/gates.py + configs/gates/g-t158.yaml      (new gate; regenerate gate_presets.py)
  api/routers/planner.py      (ext) POST /planner/train-sim
  web/static/js/pages/ternary.js, methods.js     (new pages); jobdetail.js, newjob.js, models.js (ext)
configs/ternary/*.yaml  configs/recipes/ternary-*.yaml  configs/compress/ternary-ptq.yaml   (new)
docs/TERNARY.md  docs/adr/0007-*.md  (new)    docs/KERNELS.md, CLAIMS.md, ROADMAP_STATUS.md  (ext)
scripts/bench_ternary.py (new, CUDA-graph timing)   scripts/gpu_smoke.sh, gpu_report.py (ext)
```

What is retired

- `python/tern158/ui/server.py` and `index.html` (stdlib server, single printed token, binds 0.0.0.0, exposes install/build/start endpoints): replaced by Forge routers with its auth and permission model.
- `orchestrator.py` Jobs class, pid-file and `control.json` protocol: replaced by JobManager, cancel signals and RunContext.
- `gpus.py` table: merged into the single Forge chip table (Section 8.3).
- `data.py` tokeniser/shard writer: replaced by Forge `dataset_ingest` and ShardedTokenDataset; its hard-won fixes (idempotent per-tokenizer prep, `os._exit` workaround for the HF thread crash, repo-root-relative path bug) are ported into Forge's ingest.
- TOML configs and `presets.py` writer: become YAML templates in `configs/ternary/` (optional `forge ternary import-toml` helper).
- Duplicate vast.ai scripts: folded into `scripts/vast_setup.sh`, `gpu_smoke.sh` and `gpu_report.py`.

5 Kernel plan

5.1 Three tiers under ADR 0002

Tier	Content	Gate to ship
1 Oracle (torch, CPU)	Port `ref_ops.py` (325 lines): pack/unpack, quantize (3 rules, group/row/tensor), dequantize (+blend), act_quant, gemm_fwd, gemm_bwd_data, hadamard, code_stats, SR rounding. Golden vectors generated from tern158 and from the sm_120 CUDA library.	CPU tests green; bit-exact vs tern158 reference.
2 Triton	Ternary W2A8 GEMM (in-kernel 2-bit unpack to int8, `tl.dot` int8 to int32, per-group fp32 scale flush, per-token epilogue), decode GEMV for M<=16, segment quantiser+packer (one program per row-group), fused act-quant reusing Forge `util_quant`, large-M route via dequant + cuBLAS (the proven W4 pattern; tern158 measured bf16-cuBLAS within 3-8% of mma at M>=512).	Triton-interpreter tests; offline PTX compile test; then GPU test vs oracle; perf row stays 'no' until CUDA-graph benchmark.
3 Native CUDA (optional)	tern158 `kernel/` tree moved to `forge/native/ternary`, extension renamed `forge_ternary_C`, built via extra `pip install forge-llm[ternary-cuda]` or JIT; selected through `Layout.executor='native'` only if probe passes and it beats Triton on that device.	selftest --json in gpu_report; documented build matrix (CUDA 12.8+ for sm_100/120, 12.9+ for sm_103/121).

5.2 Kernel-by-kernel disposition

tern158 kernel	Disposition	Reason
gemm mma.sync int8 (346-411 TOPS)	Triton port (T2) + keep CUDA as T3	Triton int8 dot lowers to mma.sync m16n8k32 on sm_80..120; CUDA is the performance reference and fallback.

tern158 kernel	Disposition	Reason
gemm wmma / dp4a GEMV	GEMV to Triton (decode); wmma not ported	wmma was the slow 'safety net' (186-241 TOPS); dp4a is only needed below sm_80.
bf16 dequant + cuBLAS (fwd and bwd-data)	torch (T1/T2 route), no CUDA	Exact (int8 and ternary representable in bf16); default for sm_103 and backward-data; use a persistent workspace (tern158 allocates per call).
quantize (absmean / TWN / Lloyd), flip and zero stats	Oracle + Triton segment kernel	Per-tensor path in CUDA relies on fragile double atomics through a float scratch; replace with a two-pass torch/Triton reduction.
act_quant, fused RMSNorm+quant	Reuse Forge `quantize_rowwise` / `rmsnorm_quant` after exactness check	Must match absmax/127, RNE, clamp +-127, scale = max/127 (Forge already hit a 1-ulp division trap: use `_div_rn`).
hadamard, swiglu, rope, rmsnorm_bwd	Reuse existing Forge ops / torch	Elementwise or row kernels; consolidate the 3 existing Hadamard copies into one.
cross_entropy, kl_div, adam_update, sumsq, finite_check	torch / Forge losses, optim (Triton SR exists)	Keep native only if profiling shows a bottleneck.
validate.cu backend self-test	Port logic as a Forge ternary probe	Feeds hardware descriptor and autotune cache: candidates triton-int8, native-mma, bf16-dequant, gemv.
L2 persistence window	Optional helper in native tier	Declared but a no-op in train.py today.

5.3 Required supporting changes

- **Chip capability.** `layouts/chip.ChipInfo` (what the kernels consult) has no `has_int8` (it exists only in `hw/capabilities.py`) and the catalog only gates fp8 through `_NEEDS_FEATURE`. Add `has_int8` and an `int8_tc_fast` flag (false on sm_103) and gate the ternary kernel on it, otherwise sm-number checks route B300 wrongly.
- **Spec.** `ternary_w158a8_gemm.yaml`: dtypes x=int8, w=ternary, scale=fp32, acc=int32, y=bf16; weight_bits 1.58 (stored 2); groups {32,64,128,256,row,tensor}; act per-token; min_sm 75 (80 for the native tier); `bitexact_int_paths: true`. Auto-discovered by the catalog and package-data glob.
- **Tuning.** New `dtype_cfg` tag and a ternary default-config policy modelled on `w4_tuning.py` (M 1-16 gemv, mid-M dot, large-M dequant+cuBLAS).
- **Benchmarks.** Forge's built-in bench times eagerly and under-reports small-M kernels; add `scripts/bench_ternary.py` using CUDA-graph replays over a ring of weight copies larger than L2 (the `bench_w4a16.py` method).
- **Fail-loud registration.** `ops.py` swallows import errors per family; add a test asserting `ternary_linear` is in `available_ops()`.
- **Env mapping.** TERN158_FORCE_REF to FORGE_FORCE_REF; allow_reference_fallback=false to FORGE_KERNEL_STRICT; TERN158_ARCHS to FORGE_TERNARY_ARCHS (native build only).
- **Packaging.** pyproject stays pure-python; native tier is an extra. Add any new data dirs to package-data.

5.4 Performance targets (from measured tern158 numbers, sm_120)

Shape (G=128)	tern158 mma (CUDA)	tern158 bf16 path	Triton target
M=4096, N=K=4096	346 TOPS (0.397 ms)	332 TOPS	Within 80% of CUDA mma; else keep T3
M=4096, N=11008	377 TOPS	359 TOPS	same
M=512	257 TOPS	260 TOPS	Not worse than bf16 path
M=16 (GEMV)	19.9 TOPS	21.6 TOPS	Match GEMV
M=1 (decode)	4.8 TOPS, about 600 GB/s	1.46 TOPS	Bandwidth-bound; compare GB/s

These are targets, not claims. Per Forge's rule the KPI cells in `docs/KERNELS.md` start at 'no' until a CUDA-graph benchmark on real hardware fills them. Note tern158 itself observes that at large M the int8 path is only about 35-41% of the assumed int8 peak, and that training with ternary weights is *not* faster than bf16 mixed precision: the value is memory, conversion and serving.

6 Training integration

6.1 The Method layer (what makes Forge 'all-inclusive')

A `MethodSpec` descriptor says: what the method is, which job type or stage runs it, what it needs (kernels, chips, flags), what artifact kind it produces, which diagnostics pack and metric prefix it emits, which planner Scheme it maps to, and its validation status. It is data, not control flow, so adding it does not destabilise training code. Initial catalog:

Family	Methods	Status in Forge	Action in this plan
Pretrain / full FT	bf16 AdamW; 8-bit AdamW; Lion	Exists	Register descriptors
Low-precision training	FP8 (R1), NVFP4 pilot (R2)	Exists (emulated, unvalidated)	Register; move behind quant-mode plug-in
PEFT	LoRA, DoRA, QLoRA (NF4), DPO	Exists	Register; add ternary base option
PTQ	RTN, GPTQ, AWQ, SmoothQuant, MR-GPTQ; FP8, NVFP4, MXFP4 formats	Exists	Add format 'ternary'
QAT	fq_int, fq_nvfp4; **ternary latent-weight QAT**	Partial	**Add fq_t158, ternary job**; expose generic 'qat' fine-tune method
Compression	Prune, low-rank, BitDelta, KV quant, lossless, distill (offline/on-policy)	Exists	Register; ternary as distillation student
Ternary	Scratch, convert-from-HF, finetune (full/LoRA), export/import	**Missing**	**This plan**
Reserved slots	BitDistill-style hidden/attention-relation KD, Muon/8-bit-Adam on latents, multi-plane ternary, N:M sparse, vector quantization (AQLM/QuIP#-style)	Not planned here	Catalog entries marked 'planned' so the UI shows the roadmap honestly; each needs its own plan

6.2 Refactor first: quant-mode plug-in for the precision manager

`PrecisionManager` is hard-wired to rungs R0/R1/R2 (`rung_mode` dict, `healing_rung: Literal['R0','R1']`, `metric_prefix` fp4/fp8, about 25 FP4-specific config fields). Bolting a third mode on would bloat it. Plan: introduce a small interface (`prepare, convert, apply_mode, set_step, autocast, high_precision, diagnostics, restore`) with R1 and R2 re-implemented as peers, **no behaviour change**, existing tests green, then add ternary as the third mode. Keep the class-swap mechanism (`m.__class__ = ...`) for the single-process/DDP path because it preserves parameter names, optimizer state, checkpoints and exempt-pattern logic (first/last layers, lm_head, router already match BitNet convention).

6.3 Ternary QAT (single GPU and DDP)

- **Module:** `TernaryQATLinear` subclassing the fake-quant pattern: latent weight Parameter, absmean/Lloyd scale recomputed every step (never learned), per-token int8 activation quant, STE with detached scale. Forward may use the packed GEMM path on CUDA (codes cached and re-derived after each optimizer step) or fake-quant matmul elsewhere.
- **Warm-up:** HF-style lambda blend W_eff = W + lam*(Q(W)-W), lam = min(1, step/warmup), implemented once in the QAT module.
- **Model switches:** SubLN before o_proj/down_proj and a per-group linear-kind map in `ModelConfig` so ternary models can be built from scratch (changes state_dict keys; loader name mapping updated). tern158's unused

`skip_layers_first/last` keys map onto the existing `exempt_first/last_layers` precision fields (finally implemented).

- **Optimizer:** latent-weight AdamW with fp32 or bf16 + stochastic rounding state (Forge has Triton SR kernels), WSD/two-stage schedules, wd-to-zero in cooldown. A 'ternary' optimizer wrapper in `forge/ternary/optim.py` is selected via a new `OptimConfig` option; it calls requantize and counts flips.
- **Healing:** reuse the existing healing-phase mechanism by widening the allowed healing modes.

6.4 Sharded ternary trainer (`zero_ternary`)

This is tern158's most distinctive design (row-sharded latent weights and Adam state, reduce-scatter of dW, re-quantise own shard after each optimizer step, **all-gather packed 2-bit codes** (the gather lives in nn.py requantize, not optim.py), 8x less traffic than bf16) and it does not fit DDP/FSDP. It becomes a new `ParallelConfig.strategy` value implemented in `forge/ternary/zero.py` behind narrow Trainer hooks instead of forking the Trainer:

- Hooks needed in `Trainer`: optimizer factory, post-step callback (requantise + all-gather), global-norm clip hook (scalar all-reduce of sum-of-squares, one host sync), state collect/load hooks.
- **Checkpoints:** shard-aware save/load with a rank manifest; latent shards in safetensors (not pickle); resume on a different GPU count (a tern158 feature worth keeping); the existing single `state.pt` path stays for none/DDP/FSDP.
- **Scale rule:** per-tensor scale cannot be used when sharded; the config validator rewrites to per-row and says so (tern158 did it silently, so the saved config differed from the requested one).
- **OOM policy:** tern158 auto-halves micro-batch on one GPU only; Forge adopts the same and documents the multi-GPU limitation.
- **Validation:** gloo 2-rank CPU test proving sharded == unsharded loss within tolerance (tern158 has only a smoke test), then NCCL on a multi-GPU rental (gate G1). The `nccl_test.py` collectives test becomes a probe check.

6.5 Data, distillation, LoRA

- **Data:** ternary jobs consume Forge dataset ids (content-addressed shards). The tern158 stateless seeded loader's exact-resume property is checked against Forge's loader and added if missing. Defaults that failed in the logs (relative data dir, unprepared tokenizer) become validation errors with a clear message.
- **Distillation:** use Forge's distill task (top-k logit cache, licence gate, forward/reverse KL, JSD). tern158's fused CE+KL is not needed unless profiling says so. Student = ternary QAT model; teacher = original bf16. Hidden-state/attention-relation (BitDistill-style) loss is a reserved slot.
- **LoRA on ternary:** works through `forge.models.lora.LoRALinear` once `t158` implements `dequantize()`. Merging an adapter into ternary codes breaks the ternary constraint, so adapters stay separate (or re-ternarise, QA-LoRA style). tern158's HF export silently drops LoRA; Forge's artifact keeps the adapter. Two LoRA implementations exist in Forge (models/lora.py, train/lora.py): consolidate on the former before adding a third base format.
- **LoRA memory bug:** frozen ternary layers in tern158 still allocate master/m/v/grad; the Forge module must not.

6.6 Health and diagnostics

Replace tern158's single `HealthMonitor` with a per-method diagnostics pack (`forge/methods/diagnostics_packs.py`) feeding Forge's stability and diagnostics modules and emitting `ternary/*` metrics:

Metric key	Meaning	Alarm (from tern158, TARGET until re-measured)
ternary/zero_frac (+min,max)	Share of weights at 0	healthy 40-52%; warn 60%; stop 80% ('collapse')
ternary/flip_rate (+max)	Share of codes changed per step	warn above 5% (noise-dominated); compare with ternary/pred_flip_rate_equilibrium = 0.59*sqrt(2*lr*wd)
ternary/scale_mean, scale_drift	Mean group scale and drift	new (listed in REPORT_NOTES, unimplemented in tern158)

Metric key	Meaning	Alarm (from tern158, TARGET until re-measured)
ternary/lam	Warm-up blend	info
ternary/loss_gap	Shadow bf16 loss gap (reuse `high_precision()` forward)	gate input
Loss spike	loss > 3x mean of last 20 logs	Forge SpikeDetector already covers; unify thresholds

To avoid bloating the SQLite metrics table (one row per key per step), emit per-step aggregates only (mean/min/max) and write per-layer snapshots as periodic job outputs for the heatmap. tern158's cloze probe (20 items, chance 0.25) is kept as a smoke indicator only and labelled as such; real quality evidence comes from Forge eval suites and gate g-t158.

7 Quantization, conversion, fine-tuning and serving

7.1 Linear kinds and the packed artifact

Item	Design
Packed module	`t158` (PackedLinear): buffers `qweight` uint8 [N, K/4] (2-bit, little-endian, byte-identical to tern158 int32 codes), `scales` fp32 [N, K/G] (or [1] per tensor), optional `input_scale/rot_block` via the existing _InputTransform mixin for online Hadamard (H-BitLinear). Name must not collide with `w2a16` (do not model ternary as 2-bit int: wastes 0.42 bpw and has no 1.58-bit packing).
QAT module	`fq_t158`: latent Parameter; `from_dense` produces `t158`.
Layout	`ternary2_row` (bits=2, 16 codes per word along K, reserved code 10 decodes to 0), hashed layout id registered at import so `TensorEntry.layout_id` and repack work. Optional later: base-3 5-trits-per-byte (1.6 bpw) and bitmap+sign (1.485 bpw) as export-only layouts (tern158 roadmap).
Bits per weight	2.0 packed; +0.125 bpw with G=128 fp32 scales; ideal log2(3)=1.585 shown in the Models card and planner (Scheme weight_bits_eff).
Artifact	Written by `models/loader.save_model`: manifest kind 't158' in config.json, `forge_compress.json` with layout id; scale tensors named `*.scales` so the store tags them as scale roles. Latents are *not* in the artifact; they live in checkpoints.
Loader edits	Add module to `_ensure_kind` import list; add suffixes if new buffer names are introduced (the unconsumed-tensor check at `loader.py:481` currently knows .qweight .scales .zeros .g_idx .payload .block_scales); `artifactmeta` label 'Ternary 1.58b'; `convert_packed_for_kernels` branch for KernelT158Linear.
.t158.safetensors	Converter both ways. Import re-derives latents from the ternary pattern with a loud warning (hard rounding discards the true latents; tern158 does the same).
HF-compatible export	Dequantised bf16 Llama folder as tern158 does, but also writes SubLN gains and adapters, and records that a standard HF loader will not reproduce the ternary model without SubLN support.

7.2 Recipe stages

- `ternarize`: post-training ternarisation (absmean/Lloyd, optional GPTQ-style Hessian reconstruction reusing HessianAccumulator). Also exposed as `PTQConfig.format='ternary'`. `PTQConfig.bits` is an int 2..8 and IntLinear only supports those, so ternary is special-cased rather than shoehorned.
- `ternary_qat`: short QAT/healing, delegating to the train path the way `distill mode=delegate` does. `pack` gains a ternary branch with layout round-trip validation.
- Recipes: `ternary-convert-llama-1b` (rotate optional -> ternarize -> ternary_qat+distill -> pack -> eval -> gate g-t158) and a PTQ-only variant. Cache keys require `code_version` bumps on semantic change.
- Order constraint: the existing `rotate` stage refuses already-compressed models, so Hadamard-before-ternary is allowed and the reverse is not.

7.3 Conversion from Hugging Face

tern158's converter (qkv and gate/up fusion, SubLN, Lloyd init, SQNR report, KD) only knows Llama/Mistral/Qwen2-like naming and drops Qwen2 biases with a warning. Forge already has its own model class (ForgeLM) with an HF loader. Plan: conversion operates on ForgeLM via `quantize_linears(model, 'fq_t158')`, so any architecture ForgeLM can load can be converted, with fused qkv/gate-up as optional kernel-side views rather than model rewrites. Phase P3 delivers Llama-family parity with tern158; Qwen bias/QK-norm support is a follow-up listed in the roadmap.

7.4 Serving

Forge's runner accepts any `nn.Linear`-compatible module, memory planning uses real loaded weight bytes, and the deployments router refuses to re-quantise an already quantised artifact. So a registered `t158` should load and serve with no engine change (no isinstance(nn.Linear) dependency was found, but the full forward path was not traced, so the P1 CPU serve test is the proof); the only serving work is the kernel swap (`KernelT158Linear`) and the inference simulator's quant list. tern158's own naive no-KV-cache `generate` is dropped.

8 Platform integration

8.1 Jobs, API, CLI

- **Job types** registered in `jobs/registry.py`: `ternary.train` (scratch), `ternary.convert`, `ternary.finetune`, `ternary.export`, plus `ternary.selftest` (port of selfcheck). `needs_gpu=True` inherits the CUDA gate; CPU smoke runs use the existing GPU-less path with the tiny preset. Three of these could collapse into one job with a mode field; the config split is a review item.
- Config models: pydantic v2, `extra=forbid`, `ui(group=...)` hints, so New Job renders the form automatically. tern158 `validate()` rules (dimension multiples of 16/64, group divisibility, flip-rate warning, convert lr > 2e-4 warning, hf_ token rejection in `hf_source`) become pydantic validators / warnings with defaults preserved exactly (betas [0.9, 0.98], wd_final_zero, shard_state).
- **API:** no new router for training. One addition: `POST /planner/train-sim`. If a `ternary` router is ever added it must be put into `app.py` lists and the hardcoded `API_ROOTS` in `tests/test_web_static.py`.
- **CLI:** shortcut registration for `forge ternary` (train/convert/finetune/export/selftest) in the existing loop in cli.py, with the same dotted `--set k.v=x` overrides tern158 users know; `forge ternary simulate` prints the simulator report; `forge doctor` gains a ternary-kernel check (replaces tern158 `doctor`, which is merged into probe / cuda_compat / gpu_report rather than duplicated).
- **Security:** none of tern158's UI-triggered pip-install/build/fetch endpoints are carried over; build is an explicit CLI/script step guarded by Forge's permission model.

8.2 Planner: training simulator, schemes, rules, claims

- **Training simulator:** port `simulator.py` (247 lines, pure Python) to `forge/planner/trainsim.py`, parameterised by a training *scheme* (bf16 mixed, FP8, NVFP4 fake-quant, QAT, LoRA/QLoRA, full-FT with FSDP, ternary) instead of hard-wired ternary INT8 forward. Forge has no training memory/time model today (its cost model is inference-oriented), so this benefits every method. Uses `ChipSpec`; calibration reuses the cost model's `fit_eta` and residual band instead of a single scalar. Output is labelled an estimate; it has only one data point (125M run) and must never be written to the claims ledger as measured.
- **Scheme:** `t158` in `planner/spec.py` (weight_bits 1.585, scale bits 32, group 128, act int8, family 't158', experimental) with aliases ternary/bitnet.
- **Rule:** extend `R-Q6-SUB35` or add `R-Q7-TERNARY` (training, experimental) with a `build_candidate` branch emitting ternarize/ternary_qat/pack/eval/gate; stage params validated through `load_recipe` + `STAGE_PARAM_MODELS`. Quality prior for family t158 is 'untested' with wide uncertainty.
- **Claims:** seed the ledger from REPORT_NOTES with evidence grades: trusted only for arithmetic (log2(3) = 1.585, 4 codes/byte, 0.25 B/weight); everything else (never-learn-scale, Lloyd recovers ~86% of absmean gap, flip-rate

formula, sm_103 int8 at 1/30 of FP8, 8x all-gather saving) as untested with `expectation` entries so `evaluate_claim` can later mark them reproduced/partial/contradicted from Forge runs. The tern158 hardware numbers enter as 'measured on RTX PRO 6000 Blackwell sm_120, CUDA 13.4, torch 2.14.0a0+nv26.08' and nothing else.

8.3 One chip table

tern158's `gpus.py` (~30 GPUs) and Forge's `ChipSpec/chips.py` disagree in places (e.g. B300 bf16 2250 vs 2500 TFLOPS, RTX 4090 int8 661 vs 330, RTX PRO 6000 only in tern158). Two sources would make simulator and inference plans contradict each other. Plan: Forge's table is canonical; extend it with `tdp_w`, `pcie_gbs` and the int8-throttle flag, add the missing GPUs (RTX 3060 to 5080 variants, RTX Ada / PRO Blackwell workstation, A30/A40, H100 PCIe, GB200/GB300), record which number came from datasheet vs tern158 measurement, and add a reconciliation unit test. `gpus.find()` fuzzy name matching moves with it.

8.4 Evaluation and gates

- New gate `configs/gates/g-t158.yaml` (status TARGET): suite_score_ratio vs bf16 parent, kl_p99, top1 flip, loss_gap_rel, long-context ratio, safety delta; flags `export_roundtrip_bitexact`, `kernel_matches_oracle`, `zero_density_in_band`. After adding the YAML, run `regenerate_presets()` (the embedded `gate_presets.py` copy otherwise fails the drift test). Training-time evidence enters through `EvalConfig.extra_metrics/extra_flags` exactly like g-fp4train.
- Optional `ternary_health` eval suite (zero density, per-layer scale stats, bits per weight).
- tern158's validation ladder is preserved as gates: **G0** kernels pass selftest per sm architecture; **G1** smoke run converges on 1 GPU and N GPUs with matching loss; **G2** convert-TinyLlama SQNR 5.5-7.5 dB; **G3** real run tok/s inside the calibrated simulator band.

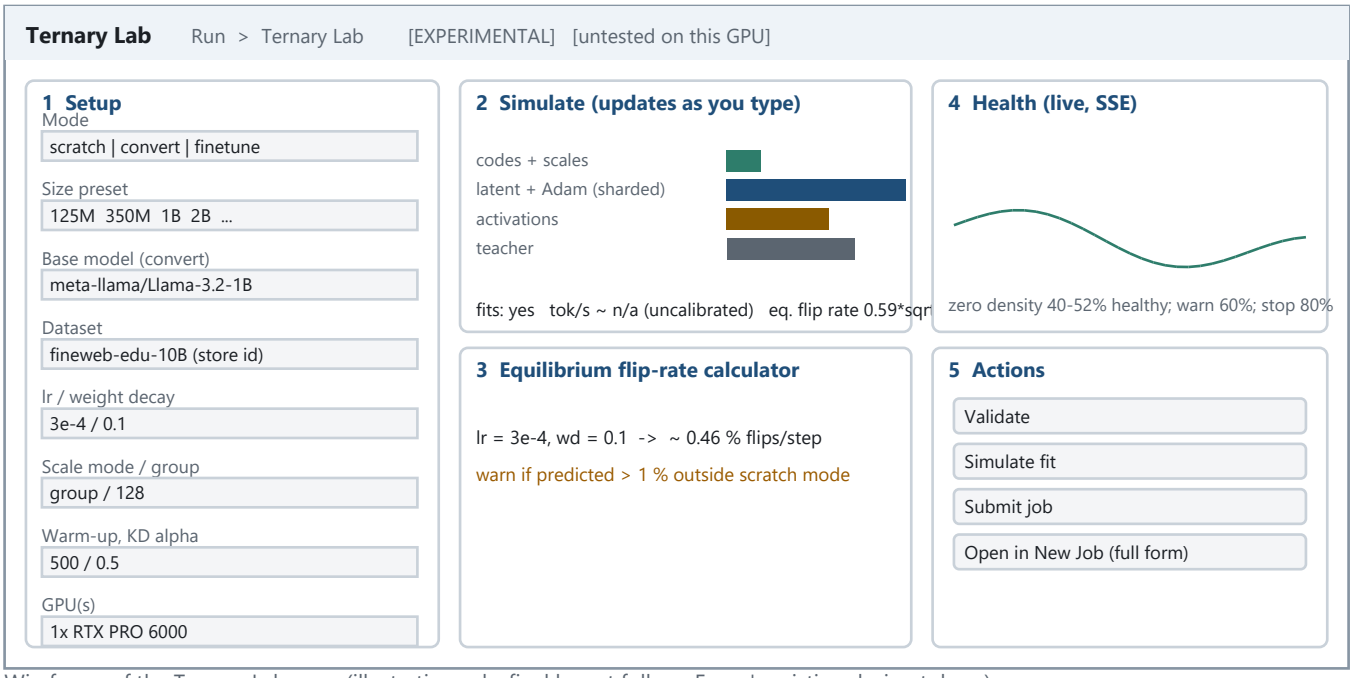
8.5 Ops tooling

- `scripts/vast_setup.sh`: optional ternary-CUDA build (needs a -devel image with nvcc; CUDA 12.8+ for sm_100/120, 12.9+ for sm_103) behind a flag; disk guidance (40 GB smoke, 300 GB real) and 'no NVLink -> grad_sync=at_step' advice move into `docs/VAST_AI.md`.
- `scripts/gpu_smoke.sh`: add the ternary smoke stage (simulate -> prepare -> 300-step train -> export). `gpu_report.py`: include native build/probe status, selftest --json, per-run log tails and tern158's token redaction (extend beyond hf_ tokens).
- Windows development: paths contain spaces; quote all paths; scripts stay LF; native build is not expected on this machine, so CPU + Triton-interpreter tests are the local gate.

9 UI additions

Constraints (ADR 0006 and `tests/test_web_static.py`): vanilla JS, no CDN or remote URLs, no build step, nav links must equal the route table, every `pages/*.js` must be routed with a default export, localStorage guarded. The UI work is mostly additive because forms are generated from pydantic schemas.

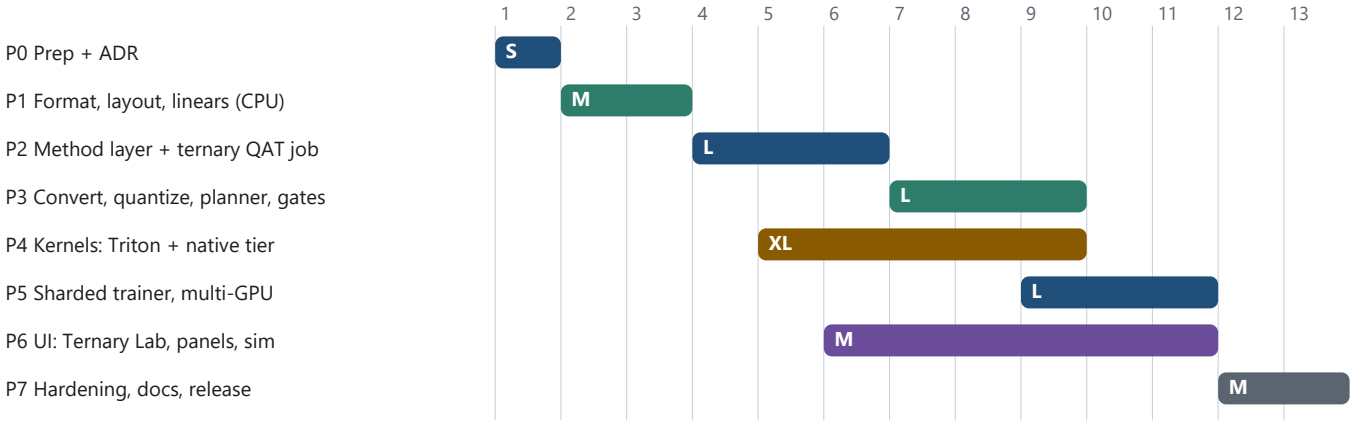
Surface	Change	Phase
New Job	`ternary.*` types appear automatically; add EXPERIMENTAL badge and template picker from `configs/ternary/`.	P2
Job detail	Generalise the hard-coded `fp4/` diagnostics panel into a list of panel descriptors `{prefix, title, badge, notes}`; add ternary panel (zero density with healthy band shading, flip rate vs predicted equilibrium, scale drift, loss gap) and exclude `ternary/` from the generic add-chart list. About 15 lines of change plus a notes map.	P2
Method picker (new page `methods.js`)	Cards from the `forge.methods` catalog grouped by family (pretrain, low-precision, PEFT, PTQ, QAT, compression, ternary) showing requirements, status (validated / experimental / planned) and a 'Start' button that opens New Job pre-filled. Honest 'planned' cards for reserved slots.	P6
Ternary Lab (new page `ternary.js`)	Wizard modelled on `finetune.js` (quick pickers via `findProp`): mode, size preset, base model, dataset, lr/wd, scale mode, group, warm-up, KD alpha; live simulator panel; equilibrium flip-rate calculator; health tab; link to full New Job form. Degrades with the standard unavailable notice if `forge.ternary` is absent.	P6
Training simulator panel	Shared component (memory-per-GPU bars, step-time breakdown, cost, advice) used by New Job and Ternary Lab for every method, calling `POST /planner/train-sim`; calibration field.	P3/P6
Models	Quantization badge and filter for 'Ternary 1.58b'; ternary model card with packed bytes, bits per weight, group size, layout id.	P1
Kernel Lab / Hardware	Ternary rows with availability reasons (needs int8 tensor cores; B300 routed to bf16 path); selected backend and probe result per device; native build status.	P4
Compare	Add flip/zero metric names to the min/max direction regex.	P2
Recipes / Planner / Inference	t158 in quant mapping and weight-bits map; ternary recipe templates.	P3
Settings / dashboard	Ternary kernel + native build status next to the CUDA compatibility banner (reuses the in-flight cudaBanner).	P4
Demo mode (`mock.js`)	Hand-written schemas: add ternary job type and synthetic ternary series, clearly labelled DEMO DATA. Also add a synthetic ternary demo job to `run_selftest` as the fp4 one does.	P2/P6



Wireframe of the Ternary Lab page (illustrative only; final layout follows Forge's existing design tokens).

10 Phased roadmap with exit criteria

Phase sequencing (relative, in work-blocks; bars overlap where phases can run in parallel)



Sizes are relative: S = a day or two, M = under a week, L = about a week or more, XL = multi-week with hardware iteration. They are planning guesses, not commitments. P3 and P4 can run in parallel after P2 (they touch disjoint files), as can parts of P6.

Phase	Scope	Exit criteria (all must hold)
P0 Prep (S)	Land CUDA-compat commit; branch `ternary-merge`; ADR 0007; docs/TERNARY.md skeleton; import tern158 source preserving origin (NOTICE + licence decision); resolve Section 14.	Clean tree; ADR approved; baseline `pytest -q` green.
P1 Format + CPU path (M)	formats/ternary.py, ref oracle, layout ternary2_row, PackedTernary, `t158` and `fq_t158` linears, loader/artifactmeta edits, .t158 converter, Models badge. Golden vectors from tern158.	Pack/unpack bit-exact incl. equality with `pack_bits(t&3, 2)`; STE grads correct; save->load->logits match; tiny ternary model serves on CPU via engine and evaluates via eval job.

Phase	Scope	Exit criteria (all must hold)
P2 Method layer + ternary QAT (L)	Precision quant-mode plug-in (no behaviour change), forge.methods catalog, TernaryQAT + optimizer + warm-up + health pack, SubLN switch, ternary.train job, configs/ternary templates, data via Forge ingest, metrics + Job detail panel, CLI, mock.	Existing precision tests unchanged; CPU smoke (tiny preset, 40 steps) loss drops >0.5, ternary/* metrics visible live in UI, checkpoint resume works, template/registry tests pass.
P3 Quantize / convert / finetune / planner (L)	ternarize + ternary_qat stages, PTQ format, HF convert on ForgeLM, ternary LoRA, recipes, g-t158 + presets, Scheme/rule/claims, trainsim + API + panel, one chip table.	Convert a tiny random HF Llama on CPU end to end; recipe validates; gate runs; simulator reproduces tern158 documented memory table within tolerance; chip reconciliation test.
P4 Kernels (XL)	Spec, Triton GEMM/GEMV/quantiser, ops_gemm launcher, KernelT158Linear, catalog/bench, chip int8 flags, ternary probe, native tier move + build extra, bench_ternary.py.	Interpreter + PTX-compile tests pass locally; on rented sm_120: kernel == oracle, Triton vs tern158 CUDA vs bf16 path table filled by CUDA-graph benchmark; native selftest 23/23 reproduced from Forge.
P5 Sharded trainer (L)	zero_ternary strategy, Trainer hooks, shard-aware checkpoints, global clip, OOM policy, nccl probe.	gloo 2-rank sharded == unsharded on CPU; then gate G1 on a multi-GPU rental (loss parity, nccl OK).
P6 UI (M)	Method picker, Ternary Lab, simulator panel, Kernel Lab/Hardware/Settings rows, polish.	test_web_static passes; manual walkthrough on desktop and narrow width.
P7 Hardening + release (M)	Hardware matrix runs (sm_80/86/89/90/100/103/120 as available), 125M rerun with final eval and bf16 baseline, docs, CHANGELOG, retire tern158 repo README to point to Forge.	G0..G3 recorded per architecture with unrun architectures explicitly 'untested'; docs validation tables updated honestly.

11 Testing and validation plan

11.1 CPU (runs on this machine)

- Port tern158's 6 CPU tests under Forge's conftest (drop the `sys.path` hack and per-file env setup): pack roundtrip, GEMM vs dequantised matmul (groups 0/128/K), Lloyd beats absmean, Hadamard involution, SR unbiasedness, tiny model trains, e2e scratch->checkpoint->resume->export->load.
- **New tests for gaps tern158 never covered:** HF convert (tiny random Llama), LoRA fine-tune + export round trip, distillation loss correctness, bf16-SR training, Hadamard on a full model, **gloo 2-rank sharded == unsharded**, config mapping, collapse-alarm triggers, golden vectors from the CUDA library, artifact manifest round trip, loader refuses mismatched layouts.
- Kernel tests: oracle properties, Triton CPU-interpreter vs oracle (int-only paths are bit-exact), offline PTX compile test, `available_ops()` includes ternary.
- Platform tests: registry/template tests (automatic via parametrisation), API e2e CPU smoke, gate-preset drift, web-static constraints, chip reconciliation, planner rule validation.

11.2 GPU (rented, via existing scripts)

Gate	Test	Pass condition
G0	Native selftest + Python selfcheck + Triton-vs-oracle per sm	All pass; per-backend TOPS recorded next to tern158's sm_120 baseline
G1	Smoke run 1 GPU vs N GPUs	Loss decreases (tern158 smoke: 7.84 -> 5.85 in 300 steps), zero density 35-50%, loss parity within noise, nccl collectives OK, export exists
G2	Convert TinyLlama, 200 steps	Mean reconstruction SQNR 5.5-7.5 dB (tern158 expectation)
G3	125M-token-scale real run vs simulator	Final val loss recorded (tern158 has only a step-500 point: ppl 141); bf16 baseline of same size; tok/s inside calibrated band

Unrun architectures stay marked 'no' in the KERNELS validation table; a result on sm_120 is never extrapolated to Hopper or sm_103.

12 tern158 defects and gaps to fix during the port

#	Finding	Fix in merge
1	`gemm_fwd` silently routes M<=16 to GEMV even if wmma/mma requested	Make the decode route explicit and reported (`last_path`)
2	bf16 backend allocates (N*K+M*K)*2 bytes per call	Persistent workspace
3	dW is a plain torch matmul on dequantised activations; backward is the bottleneck	Profile first; fused dW kernel is a later item
4	Per-tensor quantiser uses float scratch reinterpreted as double[8] with double atomicAdd	Two-pass reduction in torch/Triton
5	Config keys declared but unused: skip_layers_first/last, compile, teacher_dtype, l2_persist, devices	Implement via existing fields (exempt layers, Trainer compile, Teacher dtype) or drop
6	Frozen LoRA layers still allocate master/m/v/grad; LoRA not merged/ exported to HF	Zero-size latent tensors; adapters kept in artifact
7	Checkpoints use pickle torch.save/load, no weights_only	safetensors for latents; weights_only elsewhere
8	Resume from .t158 loses latents (warns)	Keep warning; document; prefer checkpoint resume
9	Tensor-scale silently rewritten to row under sharding	Validator error/rewrite with message, saved config equals effective config
10	Monitor thresholds ternary-specific, run on all methods; twostage schedule hard-coded 0.5/0.25	Per-method diagnostics pack; schedule parameters exposed
11	HF prepare crash at interpreter exit ('PyGILState_Release'); relative data dir failed 3x in the 125M run	Port the `os._exit` guard and absolute/keyed manifest paths into Forge ingest
12	Log redaction only matches hf_ tokens	Broaden secret scrubbing in gpu_report
13	OOM auto-retry single-GPU only	Document; multi-GPU raises with advice
14	Bundle-1 selftest/selfcheck disagreed on the group=0 scales-size contract	Single contract documented in the oracle and binding; test for it
15	Qwen2 attention bias silently dropped on convert	Warn loudly now; support bias/QK-norm as follow-up

13 Risk register

Risk	Impact	Mitigation
Triton ternary kernel underperforms hand-written mma (no ldmatrix/swizzle control)	Medium: slower serving/training fwd	Keep native tier optional; probe-based selection; performance targets in 5.4; bf16 route is within 3-8% at large M anyway
Forge GPU stack is largely unvalidated; merging two unvalidated stacks multiplies uncertainty	High	Explicit gates G0-G3; EXPERIMENTAL flags; tern158's selftest remains the first thing run on any new box
Precision-manager refactor regresses FP8/FP4 paths	High	Behaviour-preserving refactor first, existing tests as the guard, no ternary code in that PR
Sharded trainer conflicts with Trainer assumptions (rollback, clip, GradScaler, checkpoint)	High	Narrow hooks; CPU gloo equivalence test; ship P5 last behind a flag
Architecture lock-in: tern158 TernaryLM only Llama-like	Medium	Convert on ForgeLM via linear replacement; Llama parity first, then bias/QK-norm
Hardware table drift between simulator and planner	Medium	Single chip table + reconciliation test
Name/kind collisions in global registries (w2a16, kernel_ prefix classes, manifest kinds)	Medium	Reserve 't158', 'fq_t158' kinds; test registry for duplicates

Risk	Impact	Mitigation
Stale hand-maintained lists (nav, API_ROOTS, mock.js, gate presets, _ensure_kind, import_stage_modules)	Low-Medium	Checklists in Appendix D; the existing drift tests catch most
Metrics volume from per-layer stats	Low	Aggregates per step; snapshots as outputs
Licence: tern158 README says MIT 'placeholder'; Forge is Apache-2.0	Low (same owner)	Decide at P0; record in NOTICE
Windows dev box cannot build CUDA or run Triton	Medium for verification speed	CPU oracle + interpreter + stub-header syntax check (`tools/syntax_check.py`); GPU gates on rentals
Scope creep ('all-inclusive')	Medium	Method layer is metadata; reserved slots are explicitly out of scope and unscheduled

14 Open decisions for your review

#	Question	Options	Recommendation
D1	Kernel strategy	(a) Triton-only (b) Triton-first + optional native (c) native-first	(b); ADR 0007 amends 0002
D2	Job surface	(a) dedicated ternary.* jobs (b) rung/method inside train/finetune (c) both	(c), shared core
D3	One ternary job with a mode field, or separate train/convert/finetune types?	single vs split	Split types, shared config base (clearer templates); your call
D4	.t158.safetensors	drop / import-only / import+export	Import + export
D5	Native extension name and env vars	forge_ternary_C + FORGE_* mapping	As stated in 5.3
D6	Method layer scope	(a) metadata catalog (b) also refactor Trainer around it	(a) now; (b) not in this plan
D7	tern158 simulator scope	ternary-only vs all training schemes	All schemes (benefits every method)
D8	Which GPUs/rentals for validation, and budget	sm_120 only / + Hopper / + multi-GPU	sm_120 first (already have access); one multi-GPU rental for G1; Hopper if budget allows
D9	Licence of imported tern158 code	MIT notice kept / relicense Apache-2.0	Relicense under Apache-2.0 with NOTICE (same owner)
D10	Qwen bias / QK-norm and MoE ternary	in this plan / follow-up	Follow-up after Llama parity
D11	Reserved method slots (BitDistill KD, Muon, multi-plane ternary, N:M, vector quant)	show as 'planned' cards / hide	Show as planned; each needs its own plan
D12	Git strategy	subtree import with history / copy with attribution	Copy with attribution + NOTICE; keep tern158 repo archived

Appendix A File-by-file migration map

tern158 file	Lines	Destination in Forge	Action
kernel/src/gemm.cu	472	kernels/triton/gemm_ternary.py (T2); native/ternary/gemm.cu (T3)	port + keep
kernel/src/quantize.cu	399	formats/ternary.py + kernels/triton/quant_ternary.py; native copy	port + keep
kernel/src/activations.cu	308	reuse ops_util (quantize_rowwise, rmsnorm_quant, hadamard); native copy	reuse / keep
kernel/src/validate.cu	227	hw/probe ternary check + native/ternary	port logic
kernel/src/device.cu	112	hw capabilities/chips (int8_tc_fast); native copy	port rule
kernel/src/optimizer.cu, loss.cu	117 + 102	train/optim (SR), train/losses; native copy only	torch first
kernel/include/*, torch_ext/bindings.cpp	229 + 397	native/ternary (rename forge_ternary_C)	move
kernel/tests/selftest.cu	438	native/ternary/tests; invoked by gpu_report	move
CMakeLists.txt, setup.py, Makefile, tools/*	~190	native/ternary build + hw/cuda_compat archs; stub headers kept for CI	move
python/tern158/ref_ops.py	325	kernels/ref/gemm_ternary.py + formats/ternary.py	port (oracle)
python/tern158/kernels.py	141	dispatch pattern absorbed by ops_gemm.py	re-home
python/tern158/nn.py	631	ternary/qat.py, models/linear.py, ternary/zero.py (sharding)	split + port
python/tern158/optim.py	183	ternary/optim.py (+ schedules)	port
python/tern158/train.py	407	ternary/run.py + Trainer hooks	re-home
python/tern158/convert.py	165	ternary/convert.py (on ForgeLM)	port
python/tern158/checkpoint.py	119	train/checkpoint.py shard-aware path	re-home
python/tern158/export.py	140	ternary/export.py	port
python/tern158/monitor.py	138	ternary/health.py + methods/diagnostics_packs.py	split
python/tern158/config.py, presets.py	350 + 99	ternary/config.py, presets.py, configs/ternary/*.yaml	convert
python/tern158/data.py	192	train/data ingest (fixes ported)	retire after porting fixes
python/tern158/evalu.py	79	eval suite / probe (smoke only)	port
python/tern158/dist_utils.py	119	train/dist.py	absorb
python/tern158/simulator.py	247	planner/trainsim.py	port + generalise
python/tern158/gpus.py	87	hw/chips.py + planner/spec.py	merge, then delete
python/tern158/orchestrator.py, ui/*	431 + ~570	jobs/manager, probe/cuda_compat, Forge UI	retire; port doctor checks
python/tern158/selfcheck.py, nccl_test.py	218 + 44	ternary.selftest job, tests_gpu, probe check	port
python/tern158/cli.py	205	forge/cli.py shortcuts	re-home
python/tests/*	219	tests/test_ternary_*.py (+ new tests)	port + extend
scripts/*.sh	50	scripts/vast_setup.sh, gpu_smoke.sh, gpu_report.py	merge
docs/ARCHITECTURE, REPORT_NOTES, VAST_GUIDE; README/ROADMAP/CHANGELOG	~300	docs/TERNARY.md, CLAIMS.md, VAST_AI.md, ROADMAP_STATUS.md, CHANGELOG	merge

tern158 file	Lines	Destination in Forge	Action
logs/*.tar.gz	2 bundles	docs/evidence/ (kept as provenance for measured claims)	archive

Appendix B Config key mapping (TOML to pydantic)

tern158 section	Forge destination	Notes
[model] (preset, hidden, layers, heads, kv_heads, ffn, vocab, max_seq, rope_theta, act, subln, tie_embeddings, hf_source)	ModelSourceConfig + ModelConfig (+ ternary_subln, size presets)	preset table (125m..70b) retained; hf_ token rejection kept
[quant] (scale_mode, group, rule, lloyd_iters, act_hadamard, warmup_steps, skip_layers_first/last)	TernaryQuantConfig	skip_layers -> exempt_first/last_layers
[train] (mode, finetune_method, lora_*, tokens/steps, seq_len, batch, lr schedule, wd, betas, clip, dtypes, checkpointing, intervals, resume, compile)	TrainConfig / FinetuneConfig + ternary section	defaults preserved exactly; compile -> Trainer compile
[distill] (enabled, teacher, alpha, temperature, teacher_device, teacher_dtype)	DistillConfig	alpha 0.5, T 1.0 defaults kept; teacher_dtype implemented
[data] (tokenizer, tokens_to_prepare, val_tokens, shard_tokens, dir, sources)	dataset_ingest config + dataset id	no more repo-root-relative dirs
[system] (devices, shard_state, grad_sync, gemm_backend, l2_persist, tf32, allow_reference_fallback, workdir, expandable_segments)	job gpus; ParallelConfig.strategy=zero_ternary; ternary.grad_sync; kernel env switches; FORGE_HOME	l2_persist dropped unless native tier uses it
[monitor] (zero_warn/stop, flip_warn, loss_spike_factor, stop_on_nan, nvml_every_s)	ternary diagnostics pack + StabilityConfig + SystemMonitor	thresholds TARGET until re-measured

Appendix C Measured results carried over (RTX PRO 6000 Blackwell, sm_120)

Hardware: NVIDIA RTX PRO 6000 Blackwell Server Edition, 188 SMs, 97,887 MiB, driver 580.95.05, CUDA 13.4, torch 2.14.0a0+4fdf77b940.nv26.08, Python 3.12.3, single GPU. Source: two log bundles from 2026-10-27 (`logs/tern158_report_20261027_054024.tar.gz` and `..._071050.tar.gz`). Everything off this chip is untested.

Item	Result
Bundle 1 selftest	20 passed / 3 failed (per-tensor G=0 quantiser scale check in the harness, 0 code mismatches); Python selfcheck crashed on group=0 scales size. All GEMM backends passed.
Bundle 2 selftest / selfcheck	23/23 and 65/65, including a 4-layer 30-step training smoke (loss 6.578 -> 1.241). Max errors: int8 GEMM 0-1.8e-7 (fp32 out), bf16 path 2.4e-3..5.6e-3, bwd-data about 2e-3.
GEMM M=4096, N=K=4096, G=128	wmma 187, mma 346, bf16-cuBLAS 332 TOPS; per-tensor (G=0): wmma 238, mma 407.5, bf16 334
GEMM M=4096, N=11008	wmma 191, mma 377, bf16 359 TOPS
GEMM M=512 / 16 / 1	mma 257 (bf16 260) / 19.9 (GEMV; bf16 21.6) / 4.8 TOPS at about 600 GB/s
Smoke train (45M, 300 steps, 9.83M tokens)	loss 7.84 -> 5.85; val ppl 876 -> 377; probe 0.30; about 415k tok/s; zero fraction 40.4-40.9%; flip 0.54-0.69% falling to 0.10%; export 54.3 MiB
125M run 'one' (200M tokens, 763 steps)	19.7 min, about 170k tok/s, MFU about 22% (vs bf16 peak), 9.0 GB; train loss 8.67 -> 4.32; val loss 4.949 / ppl 141.1 at step 500 (only eval); probe 0.35; zero 40.4-40.5%; flip 0.06 -> 0.33 -> 0.09%
Failures	scratch_125m run crashed 3x (missing data manifest for the tokenizer); data-prep process died at exit with PyGILState_Release (data was already written).
Not covered	Multi-GPU/NCCL, sm_80/86/89/90/100/103, final eval, bf16 baseline, any conversion run, simulator calibration.

Appendix D Extension-point checklists

To add...	Touch these (and the test that guards it)
Job type	pydantic config (extra=forbid, ui hints) -> entry(cfg, ctx) -> `register_job_type` -> template YAMLs (first line a comment; EXPERIMENTAL marker) -> add dir to `JOB_DIRS` in test_configs_templates.py and name to expected set in test_integration_registry.py -> CLI shortcut + docs rows
Recipe stage	param model in `STAGE_PARAM_MODELS`, `@_stage` function with code_version, module in `import_stage_modules()`, recipe template
Gate	YAML in configs/gates, `regenerate_presets()`, test_eval_gates drift test
Linear kind	`@register_linear`, loader `_ensure_kind` list, suffix check, artifactmeta label, PACKED_CLASSES, serialize resolve order
Kernel	spec YAML, ref oracle + export, layout + register, Triton kernel, ops_gemm launcher + _DOCUMENTED, catalog feature gate, bench case, CPU + interpreter + GPU tests, KERNELS.md rows starting at 'no'
UI page	pages/x.js (default export render), routes.js row, index.html nav link, API_ROOTS if a new root, mock.js entry
Claim / rule / scheme	claims_seed `_c(...)` (idempotent seed), rulebase RULES + build_candidate branch, planner SCHEMES + aliases, quality prior with provenance
Chip	configs/chips YAML, hw/chips.py rate table, planner chip builder, runbook adding-a-chip-descriptor.md
DB table	`register_migration` with new module+version (append-only)